Bst Traversal

AlgoExpert · Medium · Binary Search Trees

The problem

- In-order, pre-order and post-order traversals appending into a list.

Intuition

- In-order, pre-order and post-order differ only in when the node is visited relative to its children.

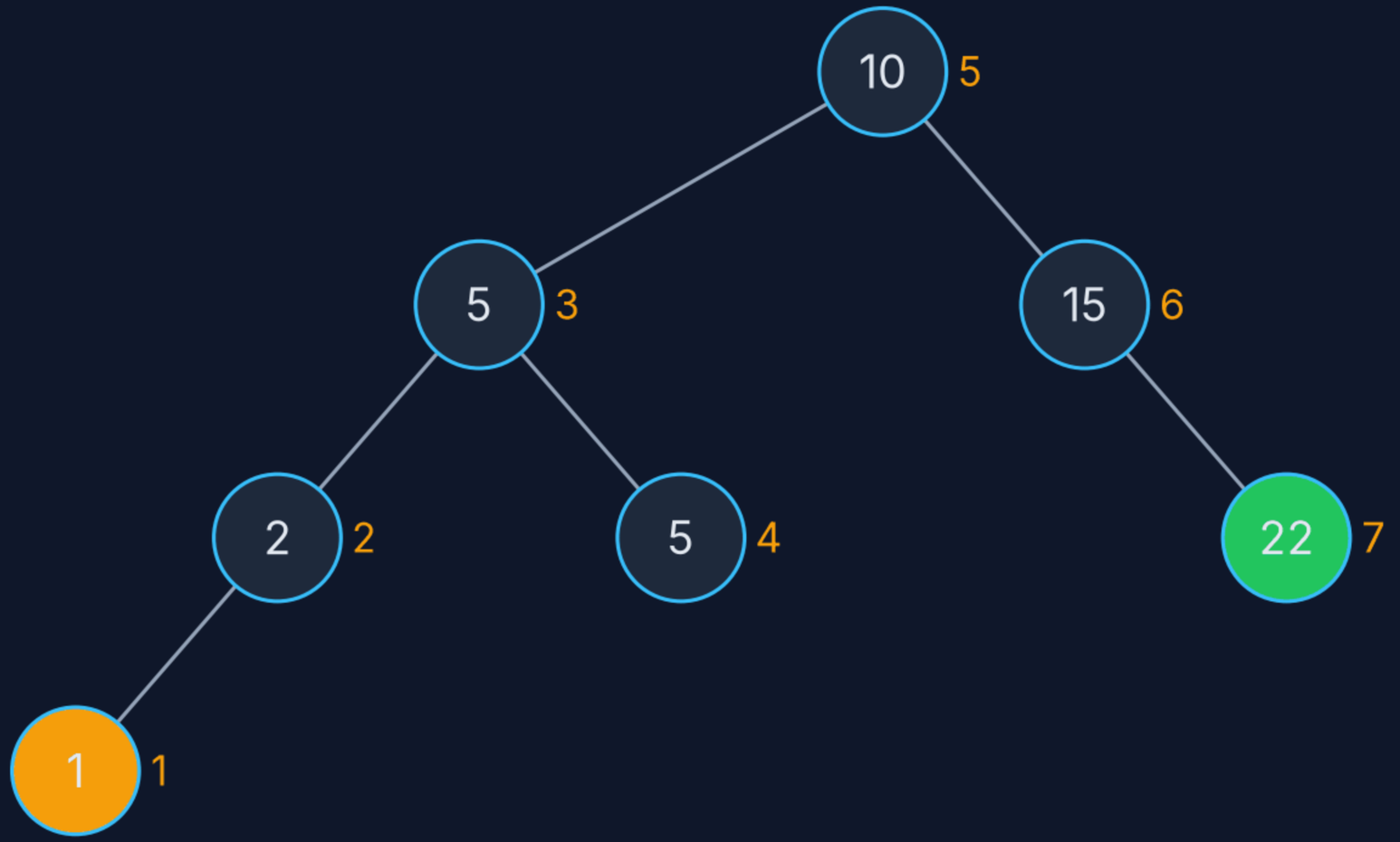
Brute force vs optimal

- All three are O(n) time, O(h) stack
- Only the position of 'visit' changes
- Iterative versions use an explicit stack

Approach

- 1. In-order: left, node, right
- 2. Pre-order: node, left, right
- 3. Post-order: left, right, node

Worked example

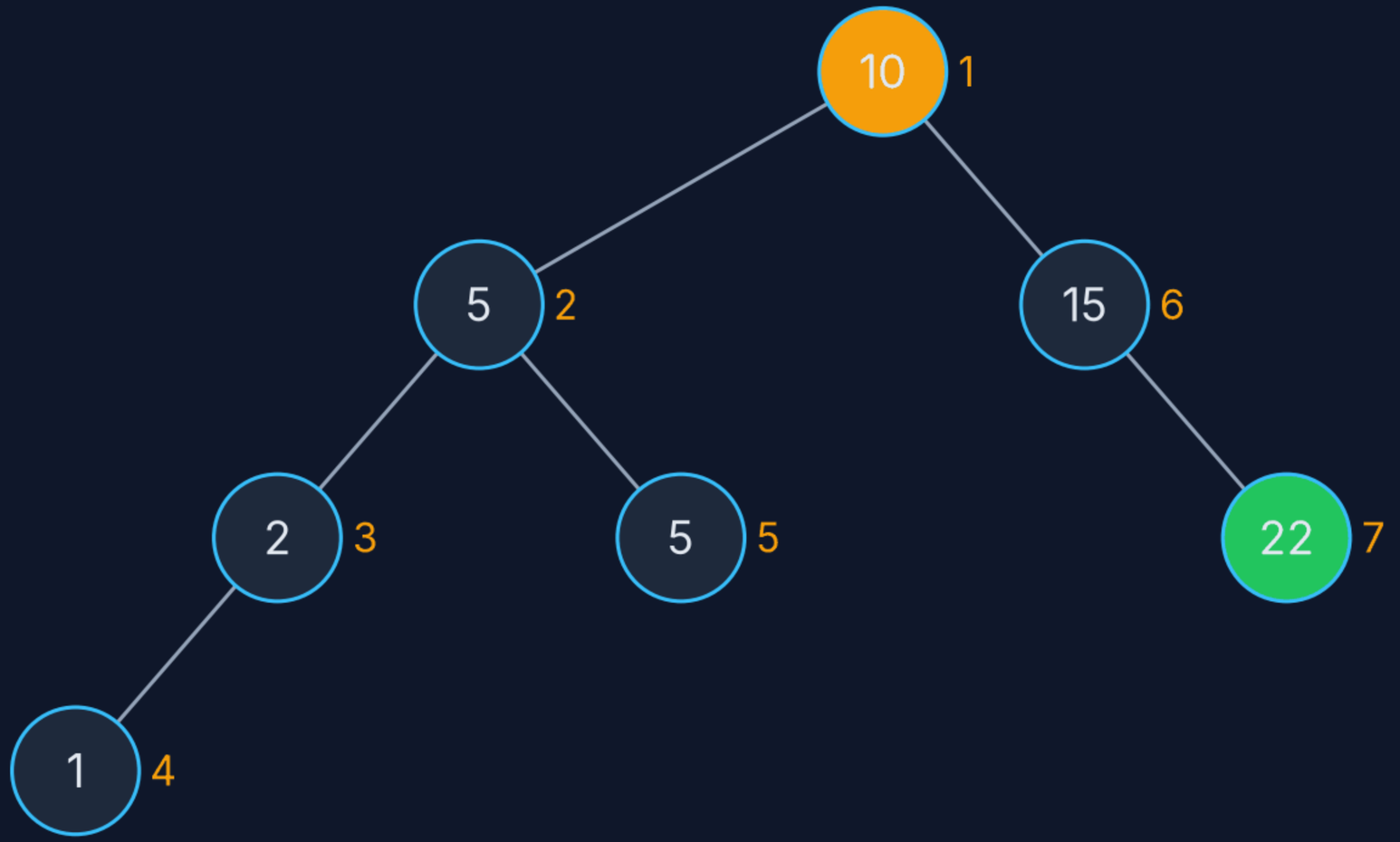


In-order

1 2 5 5 10 15 22

In-order (left, node, right): [1, 2, 5, 5, 10, 15, 22]

Worked example

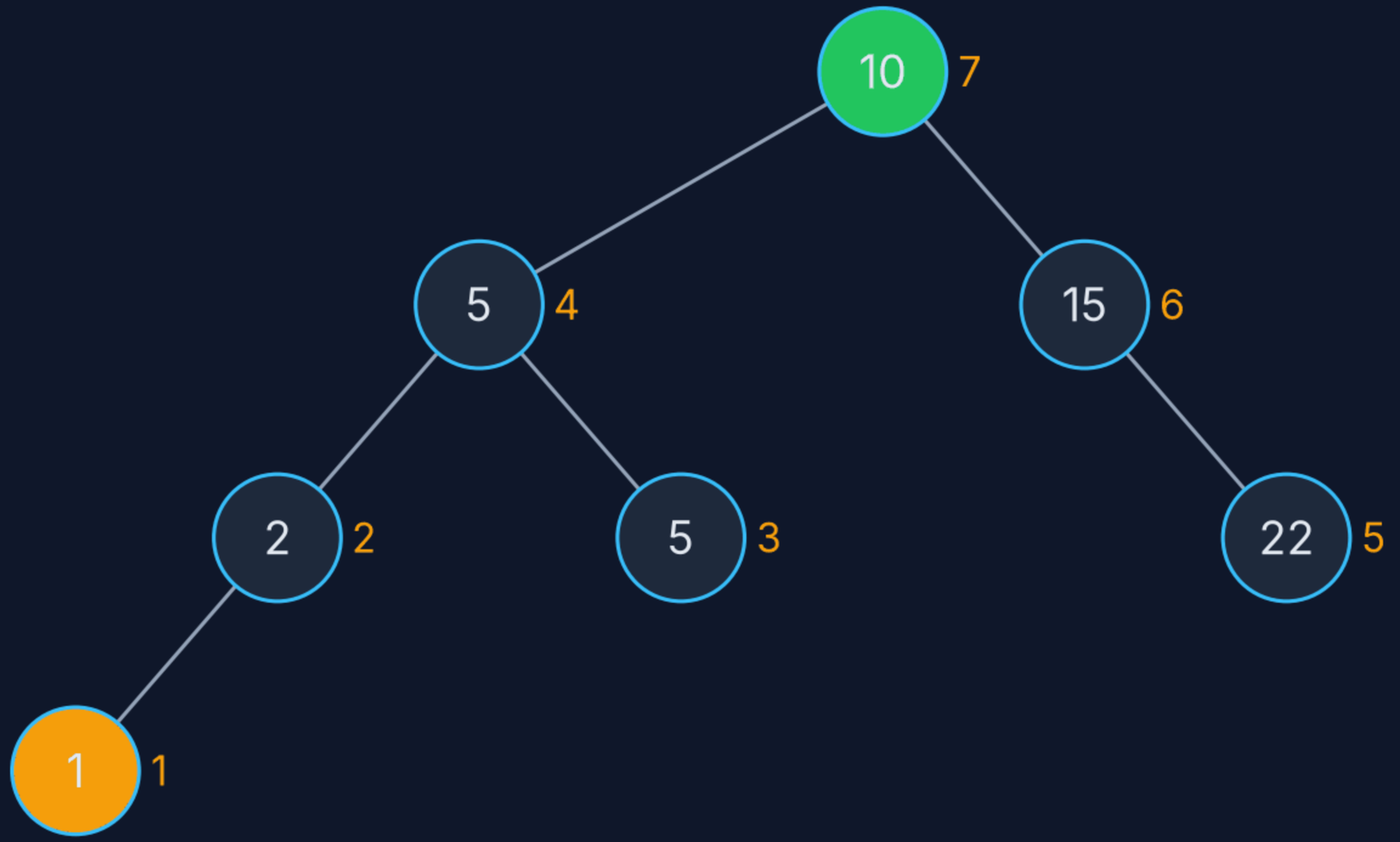


Pre-order

10 5 2 1 5 15 22

Pre-order (node, left, right): [10, 5, 2, 1, 5, 15, 22]

Worked example



Post-order

1 2 5 5 22 15 10

Post-order (left, right, node): [1, 2, 5, 5, 22, 15, 10]

Java solution

```java
public class BstTraversal {
    public static List<Integer> inOrderTraverse(BST t, List<Integer> out) {
        if (t != null) { inOrderTraverse(t.left, out); out.add(t.value); inOrderTraverse(t.right, out); }
        return out;
    }

    public static List<Integer> preOrderTraverse(BST t, List<Integer> out) {
        if (t != null) { out.add(t.value); preOrderTraverse(t.left, out); preOrderTraverse(t.right, out); }
        return out;
    }

    public static List<Integer> postOrderTraverse(BST t, List<Integer> out) {
        if (t != null) { postOrderTraverse(t.left, out); postOrderTraverse(t.right, out); out.add(t.value); }
        return out;
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- In-order of a BST = sorted order
- Empty tree → unchanged list
- Deep trees can overflow the call stack

Interview tips & follow-ups

- Pre-order serialises/copies a tree; post-order deletes/evaluates
- Follow-up: Morris traversal with O(1) space

Recap

- In-order, pre-order and post-order differ only in when the node is visited relative to its children.
- Complexity: O(n)